Day 137 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 5 reference

Watch: CGANs explained (Data Science in your pocket) | Implementation: Conditional GAN (cGAN) from Scratch with PyTorch + MNIST (CEVI)

`day137_conditional_gan.py`, segment by segment, in the order `main()` uses things. The one-hot label, the two joined inputs, the conditional round of the game and the variety measure are new; the convolutional players are Day 136's, the losses and AdamGAN are Day 134's, and the judge is Day 116's softmax regression, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: z = noise · y = label (0–9) · k = noise size (64) · G, D = generator, discriminator · n = batch size

Segment 0 — Setup

Imports: earlier lessons, kept quiet

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day126_autoencoders as d126m
    import day119_exercise as d119
    import day116_exercise as d116
    import day134_exercise as d134
    import day136_exercise as d136
```

- `with contextlib.redirect_stdout(io.StringIO()):` — inside this block, print writes into a string in memory instead of the screen: importing an exercise runs its checks, and we do not want them here.
- `import day136_exercise as d136` — the exercise files hold each day's parts verbatim, so a later day reuses exactly the code that was checked.

Segment 1 — The label, given to both players

onehot: a label as ten numbers

MATH

$$onehot(3) = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$$

CODE

```
def onehot(y, n=10):
    E = np.zeros((len(y), n), np.float32); E[np.arange(len(y)), y] = 1
    return E
```

- `np.zeros((len(y), n), np.float32)` — one row per label, ten columns, all zero.
- `E[np.arange(len(y)), y] = 1` — fancy indexing with two arrays: sets E[0, y[0]], E[1, y[1]], ... in one statement.

g_input and d_input: joining the label on

MATH

$$G \text{ input: } [z, \text{onehot}(y)] \in R^{74}; \quad D \text{ input: } [x, \text{plane}_0, ..., \text{plane}_9] \in R^{11 \times 784}$$

CODE

```
def g_input(Z, y):
    return np.concatenate([Z, onehot(y)], axis=1)
def d_input(X, y):
    P = np.zeros((len(X), 10, 784), np.float32); P[np.arange(len(X)), y] = 1
    return np.concatenate([X, P.reshape(len(X), -1)], axis=1)
```

- `np.concatenate([Z, onehot(y)], axis=1)` — side by side along the columns: 64 noise numbers then 10 label numbers per row.
- `np.zeros((len(X), 10, 784))` — ten blank 28 × 28 planes for every image.
- `P[np.arange(len(X)), y] = 1` — lights the whole plane y[i] of image i: a label as an image the convolution can read everywhere.
- `P.reshape(len(X), -1)` — -1 lets numpy work out the size: 10 × 784 = 7,840 numbers per image.

cgan_players: Day 136's players, wider at the input

CODE

```
def cgan_players(seed, k=64, c=32, d_label=True):
    G = d136.dcgan_players(seed, k=k + 10, c=c)[0]
    D = d136.dcgan_players(seed, c=c, ch=11 if d_label else 1)[1]
    return G, D
```

- `d136.dcgan_players(seed, k=k + 10, c=c)[0]` — [0] keeps the generator of the returned (G, D) pair; its Dense layer now reads 74 numbers.
- `ch=11 if d_label else 1` — a conditional expression: 11 input channels with the label planes, 1 without.

cgan_step: one round of the conditional game

MATH

$$D\text{'s loss: } -\log D(x, y) - \log(1 - D(G(z, y'), y')); \quad G\text{'s loss: } -\log D(G(z, y'), y')$$

CODE

```
def cgan_step(G, D, oG, oD, X, y, rng, k=64, d_label=True):
    n = len(X); join = (lambda I, l: d_input(I, l)) if d_label else (lambda I, l: I)
    yf = rng.randint(0, 10, n); F = G.forward(g_input(rng.standard_normal((n,
        k)).astype(np.float32), yf))
    Af = D.forward(join(F, yf)); Ar = D.forward(join(X, y))
    ld, gr, gf = d134.d_loss(Ar, Af)
    g1, _ = D.backward(gr); D.forward(join(F, yf)); g2, _ = D.backward(gf)
    oD.step(D.net, [(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)])
    yf = rng.randint(0, 10, n); F = G.forward(g_input(rng.standard_normal((n,
        k)).astype(np.float32), yf))
    lf = D.forward(join(F, yf)); lg, gl = d134.g_loss(lf)
    _, dJ = D.backward(gl); gG, _ = G.backward(dJ[:, :784])
    oG.step(G.net, gG)
    return ld, lg, float(d134.d126.sigmoid(Ar).mean()), float(d134.d126.sigmoid(lf).mean())
```

- `join = (lambda I, l: d_input(I, l)) if d_label else (lambda I, l: I)` — choose the joining function once; the rest of the step calls join without asking which version it is.

- `yf = rng.randint(0, 10, n)` — random digits to ask the generator for: fakes do not copy the batch's labels.
- `D.forward(join(F, yf)); g2, _ = D.backward(gf)` — the fake batch is run again before its backward pass, because D's caches hold the last forward (Day 136).
- `[(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)]` — the real and fake gradients summed layer by layer, weights and biases.
- `dJ[:, :784]` — D's input gradient covers the image and the label planes; only the image came from G, so only its 784 columns go back.

Segment 2 — Measuring

spread: how varied a set of images is

CODE

```
def spread(F):
    D2 = d119.sqdist(F, F); return float(np.sqrt(np.maximum(D2[np.triu_indices(len(F), 1)],
        0)).mean())
```

- `d119.sqdist(F, F)` — Day 119's squared distances between every pair of rows.
- `np.triu_indices(len(F), 1)` — the positions above the diagonal: each pair once, no image with itself.
- `np.maximum(..., 0)` — rounding can make a tiny squared distance slightly negative; clip it before the square root.

make_judge: names the digit, against the label asked for

CODE

```
def make_judge(X, y, Xt):
    jW, jb = d116.fit_softmax(X[:60000].astype(np.float64), y[:60000], 10, lam=1e-3, steps=300)
    P = lambda I: d116.softmax(I.astype(np.float64) @ jW + jb)
    def judge(F, asked):
        p = P(F); said = p.argmax(1); per = np.array([np.mean(said[asked == d] == d) for d in
            range(10)])
        near = float(np.sqrt(np.maximum(d119.sqdist(F[:500], Xt).min(1), 0)).mean())
        within = np.mean([spread(F[asked == d][:100]) for d in range(10)])
        return float(np.mean(said == asked)), float(per.min()), float(p.max(1).mean()), near,
            float(within), per, said
    return judge, P
```

- `lambda I: d116.softmax(I.astype(np.float64) @ jW + jb)` — a function stored in a variable: the judge's ten probabilities for each image.
- `def judge(F, asked):` — a function defined inside a function: it can use jW, jb and Xt from the outer one (a closure).
- `said[asked == d] == d` — a boolean mask picks the images asked to be d; == d checks each one was named d.
- `return float(...), float(...), ..., per, said` — a tuple of seven results; callers pick by position, e.g. j[0] is the share drawn as asked.

Segment 3 — Training

train_cgan: training, watched every epoch

CODE

```
def train_cgan(X, y, judge, epochs, seed, d_label=True, k=64, batch=128, keep=()):
    G, D = cgan_players(seed, k, d_label=d_label)
    oG, oD = d134.AdamGAN(G.net, 2e-4, 0.5), d134.AdamGAN(D.net, 2e-4, 0.5)
    rng = np.random.RandomState(seed); n = len(X)
    Zfix = np.random.RandomState(9).standard_normal((2000, k)).astype(np.float32); yfix =
        np.tile(np.arange(10), 200)
    log, snaps, t0 = [], {}, time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(n); hs = []
        for s in range(0, n - batch + 1, batch):
            b = o[s:s + batch]; hs.append(cgan_step(G, D, oG, oD, X[b], y[b], rng, k, d_label))
        F = d136.net_generate(G, g_input(Zfix, yfix)); m = np.mean(hs, 0)
        log.append((ep, m[2], m[3], judge(F, yfix), time.perf_counter() - t0))
        if ep in keep:
            snaps[ep] = F.copy()
    return G, D, log, snaps
```

- `np.tile(np.arange(10), 200)` — 0, 1, ..., 9 repeated 200 times: interleaved, so every batch of 500 asks for every digit.
- `range(0, n - batch + 1, batch)` — start of every full batch; the last partial batch is skipped.
- `np.mean(hs, 0)` — average each of the four numbers cgan_step returned, over the epoch's batches.
- `snaps[ep] = F.copy()` — .copy(), so a later epoch's images cannot overwrite these.

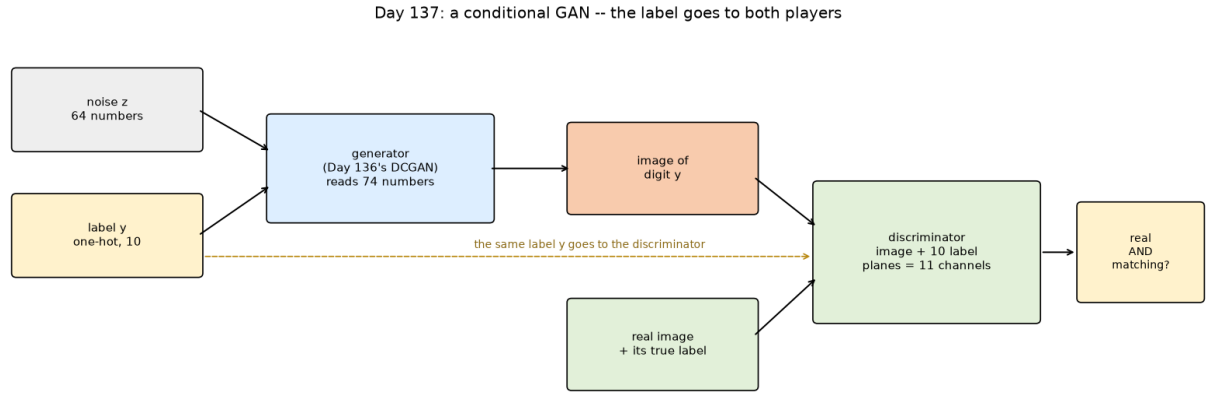
print_log: one table per run

CODE

```
def print_log(name, log, epochs):
    print(f"{name}  {'epoch':>5} {'D(real)':>8} {'D(fake)':>8} {'asked digit drawn':>18} {'worst
        digit':>12} {'confidence':>11} {'nearest':>8} {'variety':>8}")
    for ep, dr, df, j, _ in log:
        if ep in (1, 2, 3, 5, 10, 15, epochs):
            print(f"{'':{len(name)}s}  {ep:5d} {dr:8.3f} {df:8.3f} {j[0]:18.3f} {j[1]:12.3f}
                {j[2]:11.4f} {j[3]:8.2f} {j[4]:8.2f}")
    print(f"{'':{len(name)}s}  ({log[-1][-1]:.0f} s); per digit 0-9: {' '.join(f'{v:.2f}' for v in
        log[-1][3][5])}")
```

- `f"{'':{len(name)}s}"` — a nested format: an empty string padded to the run name's length, to line the rows up under it.
- `{dr:8.3f}` — 8 characters wide, 3 decimals.

SYSTEM DIAGRAM



The label goes to both players.

Segment 4 — The experiments

demo_cgan: the confusion table

CODE

```
    G, D = runs[0][:2]; Xr, yr = X[60000:62000], y[60000:62000]; wrong = (yr + 1 +
        np.random.RandomState(3).randint(0, 9, len(yr))) % 10
    dr = d134.d126.sigmoid(D.forward(d_input(Xr, yr))).mean(); dw =
        d134.d126.sigmoid(D.forward(d_input(Xr, wrong))).mean()
    print(f"seed 0's discriminator on 2,000 real test digits: D = {dr:.3f} with the true label,
        {dw:.3f} with a wrong one")
    said = runs[0][2][-1][3][6]; asked = np.tile(np.arange(10), 200); C = np.zeros((10, 10), int)
    for a, s in zip(asked, said):
        C[a, s] += 1
```

- `(yr + 1 + randint(0, 9)) % 10` — adds 1 to 9 and wraps round: a label that is never the true one.
- `runs[0][2][-1][3][6]` — run 0 → its log → the last epoch → the judge's results → what it said for each image.
- `C[a, s] += 1` — count each (asked, seen) pair: the diagonal is the digits drawn as asked.

demo_g_only: the discriminator never sees the label

CODE

```
def demo_g_only(Xt, yt, judge, epochs=20):
    print("\n" + "=" * 70)
    print("3. THE LABEL GIVEN TO THE GENERATOR ONLY -- the discriminator sees the image, never the
        label")
    print("=" * 70)
    G, D, log, snaps = train_cgan(Xt, yt, judge, epochs, 0, d_label=False, keep=(epochs,))
    print_log("generator-only label, seed 0:", log, epochs)
    W = G.net[0][0]; print(f"size of the first layer's weights: noise rows
        {np.abs(W[:64]).mean():.4f} on average, label rows {np.abs(W[64:]).mean():.4f}")
    print("Actual output from this lesson's run.")
    return log, snaps, G
```

- `train_cgan(..., d_label=False, ...)` — the same training, with D reading the image only.
- `W[:64], W[64:]` — the first layer's rows for the 64 noise numbers and for the 10 label numbers.

demo_style: same noise, ten labels

MATH

$$slant = \Sigma\, I\,(r - r_c)(c - c_c) \,/\, \Sigma\, I\,(r - r_c)^2 \quad \textit{(column change per row)}$$

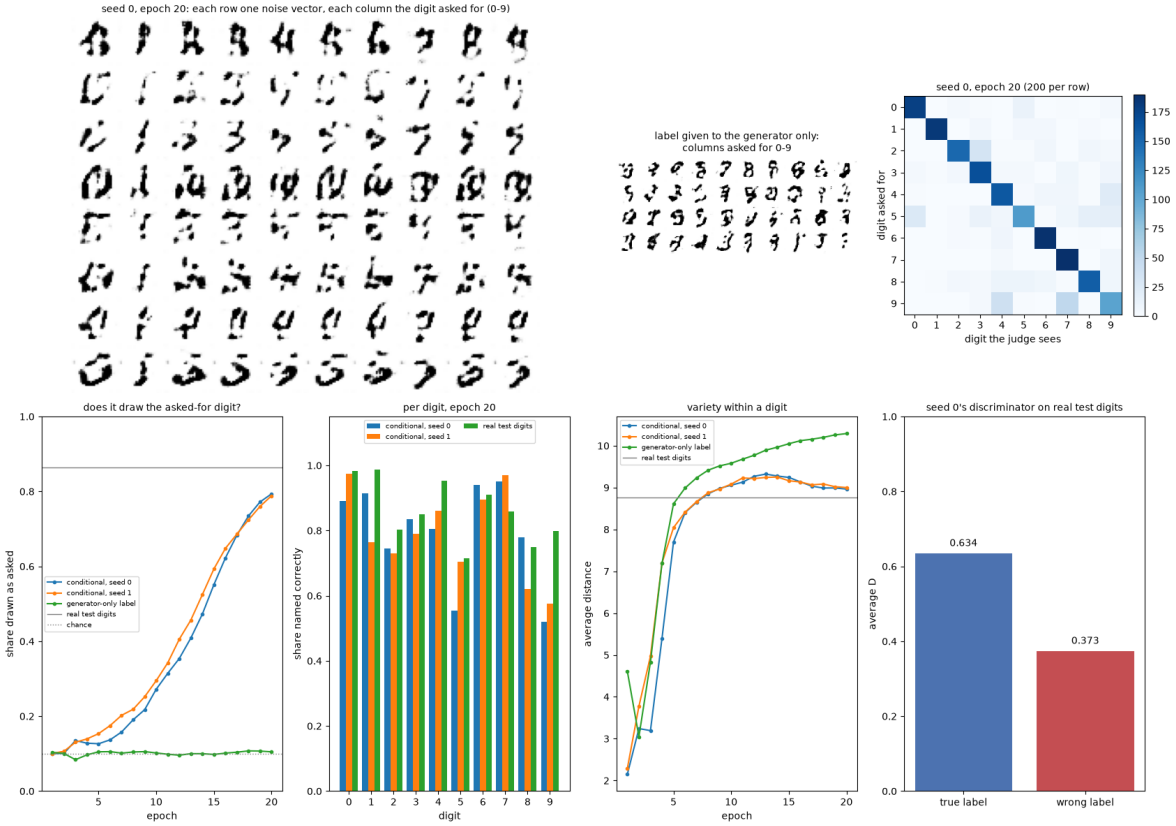
I = pixel ink, r, c = row and column, r_c, c_c = the ink's centre (rbar, cbar in the code)

CODE

```
def demo_style(runs, P):
    print("\n" + "=" * 70)
    print("4. SAME NOISE, TEN LABELS -- does the noise carry the style?")
    print("=" * 70)
    G = runs[0][0]; Z = np.random.RandomState(11).standard_normal((500, 64)).astype(np.float32)
    lab = np.tile(np.arange(10), 500); F = d136.net_generate(G, g_input(np.repeat(Z, 10, axis=0),
        lab))
    I = F.reshape(500, 10, 28, 28).transpose(1, 0, 2, 3); P_ = P(F).argmax(1).reshape(500, 10).T
    imgs = {d: I[d, :8].reshape(8, 784) for d in range(10)}
    ink = I.mean((2, 3))
    r = np.arange(28)[None, None, :, None] - 13.5; c = np.arange(28)[None, None, None, :] - 13.5;
        m = I.sum((2, 3)) + 1e-8
    rbar, cbar = (I * r).sum((2, 3)) / m, (I * c).sum((2, 3)) / m
    cov = (I * (r - rbar[..., None, None]) * (c - cbar[..., None, None])).sum((2, 3)) / m
    slant = cov / ((I * (r - rbar[..., None, None]) ** 2).sum((2, 3)) / m)
    cor = lambda A: np.mean([np.corrcoef(A[a], A[b])[0, 1] for a in range(10) for b in range(a +
        1, 10)])
    sh = np.random.RandomState(4).permutation(500)
    print("500 noise vectors, each drawn as all ten digits (seed 0's generator at epoch 20);")
    print("correlation, across the 500 noise vectors, of a measure on digit a with the same
        measure on digit b, averaged over the 45 pairs:")
    print(f"   ink (average pixel):            same noise {cor(ink):.3f};  noise shuffled for one
        digit {np.mean([np.corrcoef(ink[a], ink[b][sh])[0, 1] for a in range(10) for b in range(a
        + 1, 10)]):.3f}")
    print(f"   slant (column change per row):  same noise {cor(slant):.3f};  noise shuffled for
        one digit {np.mean([np.corrcoef(slant[a], slant[b][sh])[0, 1] for a in range(10) for b in
        range(a + 1, 10)]):.3f}")
    acc = np.mean(P_ == np.arange(10)[:, None])
    print(f"the same 500 noise vectors asked for each digit in turn: the judge names the asked-for
        digit {acc:.3f} of the time")
    print("Actual output from this lesson's run.")
    return imgs, cor(ink), cor(slant)
```

- `np.repeat(Z, 10, axis=0)` — each noise vector 10 times in a row: z_0, z_0, ..., z_1, ...; paired with labels 0-9 repeated.
- `.reshape(500, 10, 28, 28).transpose(1, 0, 2, 3)` — back to [digit, noise vector, row, column].
- `np.arange(28)[None, None, :, None] - 13.5` — row positions, shaped to broadcast against the images; 13.5 is the centre.
- `cov / (row variance)` — the slope of column against row inside the ink: its slant.
- `np.corrcoef(A[a], A[b])[0, 1]` — the correlation between two lists; corrcoef returns a 2 × 2 matrix, [0, 1] is the one wanted.
- `ink[b][sh]` — the same digit's values in shuffled order: a different noise vector against each one, the baseline.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `E[np.arange(n), y] = 1` — fancy indexing with two arrays: one entry per row.
- `np.concatenate([A, B], axis=1)` — join arrays side by side.
- `f if cond else g` — a conditional expression: choose a value (here, a function).
- `def outer(): def inner(): ...` — a closure: the inner function remembers the outer one's variables.
- `np.triu_indices(n, 1)` — positions above the diagonal of an n × n matrix.
- `np.repeat / np.tile` — repeat each element / repeat the whole array.

End of Day 137 syntax reference. For the experiments and what they showed, see the Day 137 complete guide.